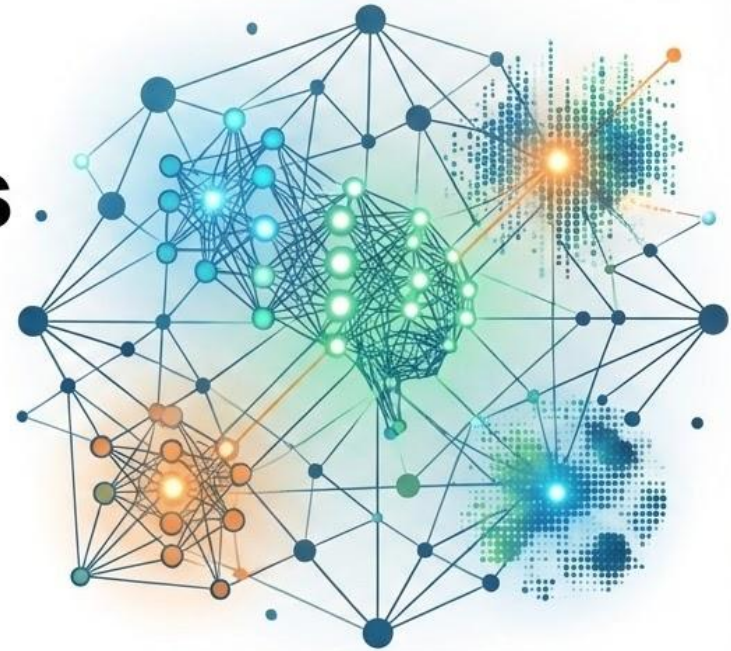


Diplomatura en python para la ciencia de datos y machine learning

Módulo 1 – Desarrollo
Avanzado en Python

3- Funciones y Modularidad



Funciones

Definición y Llamada

Una función es un bloque de código reutilizable que realiza una tarea específica. Se define con la palabra clave `def`.

```
def mi_funcion(parametro):  
    # Cuerpo de la función  
    print(f"Hola, {parametro}!")
```

Palabra clave
de definición

Parámetro
(información de entrada)

Para ejecutar la función, se la "llama" por su nombre, pasando los argumentos necesarios.

```
mi_funcion("Mundo")
```

Argumento
(valor real)

Valores de Retorno (`return`)

La sentencia `return` finaliza la función y opcionalmente devuelve un valor al punto de llamada.

```
def sumar(a, b):  
    resultado = a + b  
    return resultado # Devuelve el valor  
    print("Esto no se ejecutará")
```

Devuelve el valor y
sale de la función

El valor devuelto puede ser almacenado en una variable para su uso posterior.

```
suma_total = sumar(5, 3)  
print(suma_total) # Salida: 8
```

La variable captura
el valor devuelto

Ejercicio sobre funciones

Enunciado del Problema

Crea una función llamada 'potencia' que tome dos valores numéricos como argumentos (base y exponente). Deberá devolver el número que resulte de resolver la potencia, utilizando el primer número como base y el segundo como exponente. Luego ejecuta esa función.



Solución



Valores de Retorno Múltiples

En Python, **una función puede devolver múltiples valores separándolos por comas**. Estos valores se devuelven como una tupla.



```
def operacion(a, b):  
    suma = a + b  
    producto = a * b  
    return suma, producto
```

```
# Llamada y desempaquetado  
resultado_suma, resultado_producto = operacion(5, 3)  
print(f"Suma: {resultado_suma}, Producto: {resultado_producto}")
```

Paso de argumentos a una función

Argumentos posicionales: se asocian a los parámetros de la función en el mismo orden que aparecen en la definición de la función.



```
#definicion
def sumar(a, b):
    return a + b
#invocacion argumentos posicionales
sumar(5, 3) # a=3 b=5
```

Argumentos nominales: se indica explícitamente el nombre del parámetro al que se asocia un argumento de la forma **parámetro=argumento**

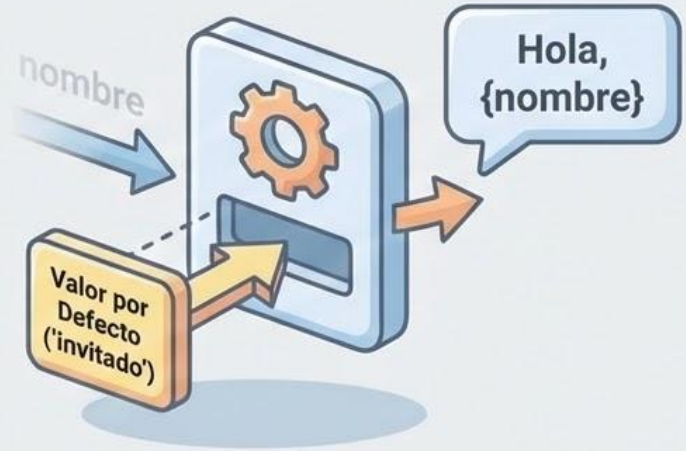


```
#definicion
def sumar(a, b):
    return a + b
#invocacion argumentos nominales
sumar(b=3,a=5)
```

Parámetros con valor por defecto

Es posible asignar un valor si no se proporciona uno al llamar a una función.

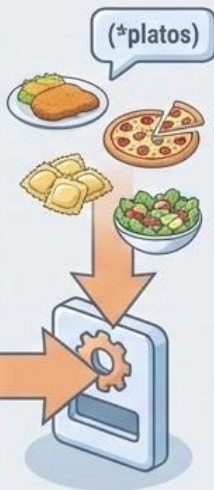
```
def saludar(nombre="invitado"):  
    print(f"Hola, {nombre}")  
  
saludar()           # Hola, invitado  
saludar("Ana")      # Hola, Ana
```



Paso de un número indeterminado de argumentos

***args:** se antepone un asterisco al nombre del parámetro y en la invocación de la función se pasa el número variable de argumentos separados por comas.

```
def menu(*platos):  
    print("Menú del Restaurante: ")  
    if not platos:  
        print("No hay platos disponibles hoy.")  
    else:  
        for i, plato in enumerate(platos, start=1):  
            print(f"{i}. {plato}")  
  
    print("Gracias por elegirnos")  
  
# invocacion  
menu("Milanesa con papas",  
      "Pizza napolitana",  
      "Ravioles de ricota",  
      "Ensalada César")
```



****kwargs:** se antepone dos asterisco al nombre del parámetro y en la invocación de la función se pasa el número variable de argumentos por pares nombre=valor, separados por comas. (se guardan en un diccionario)

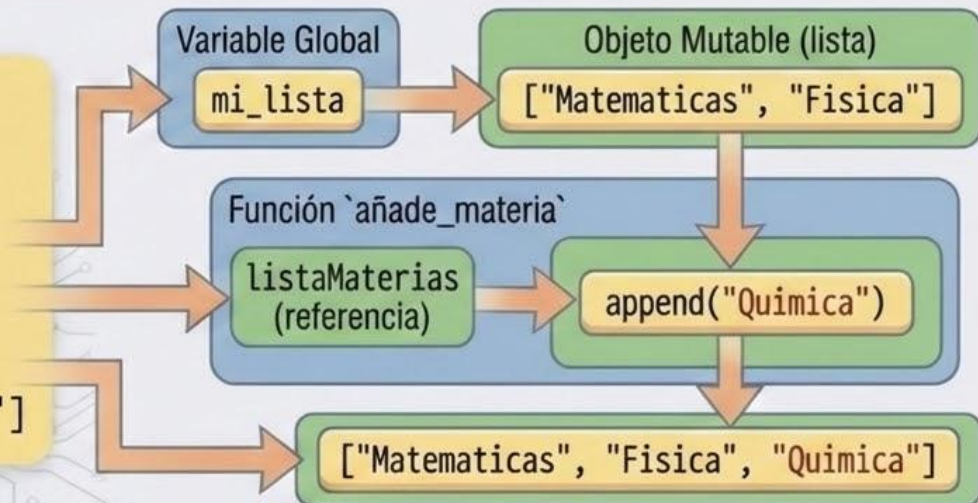
```
def menu(**platos):  
    print("Menú del Restaurante")  
    if not platos:  
        print("No hay platos disponibles hoy.")  
    else:  
        for nombre, precio in platos.items():  
            print(f"- {nombre.title()}: ${precio:.2f}")  
  
    print("Gracias por elegirnos")  
  
# invocacion  
menu(  
    milanese_con_papas=2500,  
    pizza_napolitana=3000,  
    ravioles_de_ricota=2800,  
    ensalada_cesar=2200  
)
```



Paso de Argumentos en Python (Call-by-Object-Sharing)

En Python, **no se pasa por valor** ni **por referencia** tradicional, sino por **asignación de objeto**. El efecto depende de la mutabilidad: si el objeto es mutable (como una lista), los cambios internos afectan al original; si es inmutable (como un entero), no.

```
def añade_materia(listaMaterias, materia):  
    listaMaterias.append(materia)  
    return  
  
mi_lista = ["Matematicas", "Fisica"]  
añade_materia(mi_lista, "Quimica")  
print(mi_lista)  
# Salida: ['Matematicas', 'Fisica', 'Quimica']
```



Nota: Si fuera un número o un string, el objeto original no cambiaría.

Ámbito de los parámetros y variables de una función



GLOBAL: Fuera de todas las funciones

Las variables definidas en el cuerpo principal del programa son accesibles desde cualquier parte.

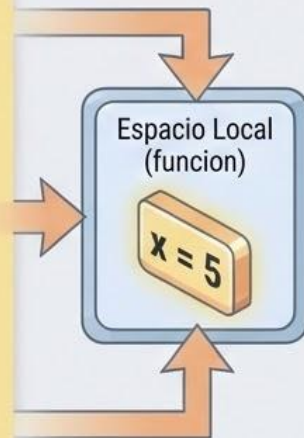
```
x = 10 # global  
  
# Accesible aquí y en  
# funciones que no la redefinen  
  
print(f"Desde fuera: {x}")  
# Salida: Desde fuera: 10
```



LOCAL: Dentro de la función

Las variables definidas dentro de una función solo existen mientras la función se ejecuta.

```
def funcion():  
    x = 5 # Local, oculta a la  
    global  
    print(f"Desde dentro: {x}")  
    # Salida: Desde dentro: 5  
  
funcion()  
print(f"Después de la función:  
{x}")  
# Salida: Después de la función:  
10
```



Funciones como objetos

Las funciones pueden asignarse a variables, pasarse como argumentos y devolverse desde otras funciones.

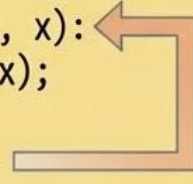
1. Asignar a variables:

```
def saludar():  
    return "Hola";  
  
mi_funcion = saludar;  
  
print(mi_funcion())  
  
# Salida: Hola
```



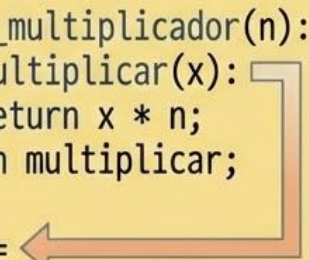
2. Pasar como argumentos:

```
def aplicar(func, x):  
    return func(x);  
  
def cuadrado(n):  
    return n * n;  
  
print(f"Resultado:  
      {aplicar(cuadrado, 5)}")  
  
# Salida: Resultado: 25
```



3. Devolver desde funciones:

```
def crear_multiplicador(n):  
    def multiplicar(x):  
        return x * n;  
    return multiplicar;  
  
duplicar = crear_multiplicador(2);  
  
print(duplicar(5))  
# Salida: 10
```



Documentación y Type Hints

Los docstrings documentan el propósito y uso de una función, mientras que los type hints (pistas de tipo) especifican los tipos de datos esperados, mejorando la legibilidad y el mantenimiento.

1. Docstrings (Documentación):

```
def calcular_area(base, altura):  
    """  
    Calcula el área de un rectángulo.  
  
    Args:  
        base (float): La base del  
        rectángulo.  
        altura (float): La altura del  
        rectángulo.  
  
    Returns:  
        float: El área calculada.  
    """  
    return base * altura
```

2. Type Hints (Tipado):

```
def saludar(nombre: str, edad: int) -> str:  
    mensaje = f"Hola {nombre}, tienes {edad}  
    años."  
    return mensaje  
  
# Uso:  
# saludar("Ana", 30) # Correcto  
# saludar("Ana", 30) # Correcto  
# saludar(123, "30") # Incorrecto (marcado  
# por herramientas)
```

3. Ejemplo Combinado:

```
from typing import List  
  
def promedio(numeros: List[float]) -> float:  
    """  
    Calcula el promedio de una lista de  
    números.  
  
    Args:  
        numeros (List[float]): Lista de  
        valores flotantes.  
  
    Returns:  
        float: El promedio de los valores.  
    """  
    if not numeros:  
        return 0.0  
    return sum(numeros) / len(numeros)
```

Introducción a Programación Funcional

Paradigma que trata el cálculo como la evaluación de funciones matemáticas y evita el cambio de estado y los datos mutables. Python soporta características clave.

1. Funciones Lambda (Anónimas): 2. Función map():

Pequeñas funciones sin nombre, definidas en una línea con ``lambda``.

```
# Función lambda para duplicar
duplicar = lambda x: x * 2
print(duplicar(5))
# Salida: 10
```

Aplica una función a cada ítem de un iterable y devuelve un mapa (iterador).

```
numeros = [1, 2, 3, 4]
# map con lambda para cuadrados
cuadrados = map(lambda x: x**2,
numeros)
print(list(cuadrados))
# Salida: [1, 4, 9, 16]
```

3. Función filter():

Filtra elementos de un iterable para los que una función devuelve True.

```
numeros = [1, 2, 3, 4, 5, 6]
# filter para números pares
pares = filter(lambda x: x % 2
== 0, numeros)
print(list(pares))
# Salida: [2, 4, 6]
```

Funciones Lambda

Las funciones lambda son funciones pequeñas y anónimas definidas con la palabra clave 'lambda'. Son útiles para operaciones cortas y a menudo se usan dentro de otras funciones.

1. Sintaxis básica:

```
# Tradicional
def cuadrado(n):
    return n * n

# Lambda
cuadrado_lambda = lambda n:
n * n
print(cuadrado_lambda(5))
# Salida: 25
```

2. Con múltiples argumentos:

```
# Lambda con varios argumentos
sumar = lambda a, b, c: a + b
+ c

resultado = sumar(10, 20, 30)
print(resultado)
# Salida: 60
```

3. En funciones de orden superior:

```
# Usando lambda con map
numeros = [1, 2, 3, 4]

cuadrados = list(map(lambda
x: x**2, numeros))

print(cuadrados)
# Salida: [1, 4, 9, 16]
```

Módulos en Python: import, from ... import y alias (as)

Los módulos son archivos que contienen definiciones y declaraciones de Python. Se usan para organizar el código y reutilizar funciones, clases y variables en diferentes programas.

1. Importación básica (import):

```
import math
```

```
# Usar el nombre del módulo  
# para acceder a sus elementos  
resultado = math.sqrt(16)  
print(resultado)  
# Salida: 4.0
```

2. Importar elementos específicos (from ... import):

```
from math import pi, sqrt
```

```
# Acceder directamente  
# a los elementos importados  
print(pi)  
print(sqrt(25))  
# Salida: 3.14159...  
# Salida: 5.0
```

3. Usar alias (as):

```
import pandas as pd  
from math import factorial as fact
```

```
# Usar el alias para el módulo  
df = pd.DataFrame({'A': [1, 2]})  
  
# Usar el alias para la función  
print(fact(5))  
# Salida: 120
```

El bloque `if __name__ == '__main__':`

Este bloque condicional permite que cierto código se ejecute solo cuando el archivo se ejecuta directamente como un script, y no cuando se importa como un módulo.

1. Archivo `mi_modulo.py`:

```
def funcion_util():  
    print("Función útil ejecutada.")  
  
print("Este código se ejecuta siempre.")  
  
if __name__ == "__main__":  
    # Este bloque SÓLO se ejecuta si  
    # se corre este archivo directamente.  
    print("Ejecutando mi_modulo.py directamente.")  
    funcion_util()
```

2. Importación y Ejecución:

```
# Archivo: programa_principal.py  
import mi_modulo  
  
print("Programa principal iniciado.")  
mi_modulo.funcion_util()
```

```
$ python mi_modulo.py  
Este código se ejecuta siempre.  
Ejecutando mi_modulo.py directamente.  
Función útil ejecutada.
```

```
$ python programa_principal.py  
Este código se ejecuta siempre.  
Programa principal iniciado.  
Función útil ejecutada.
```